Covid-19 Chest X-Ray Classification

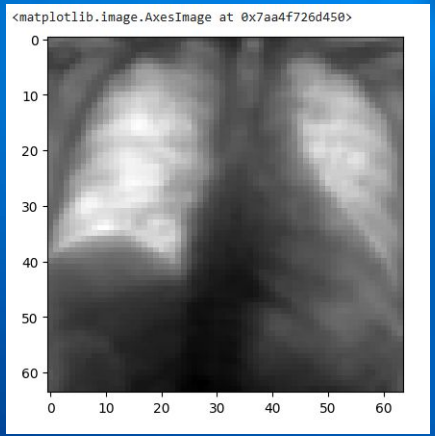
By: Lakshitha Potini, Saanvi Rangarajan, Nitish Sharma

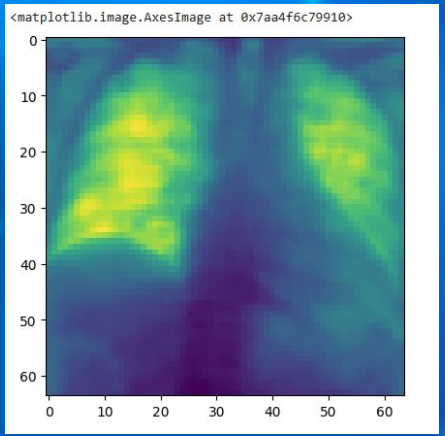
Background

The goal of this project is to classify chest x-rays & radiograms based on whether or not they have COVID-19.

According to Cleveland Clinic, COVID-19 infections can cause lung damage that leads to pneumonia, acute respiratory distress syndrome (ARDS) or scarring.

Data that we were given included 6334 images of lung x-rays, with each picture of dimensions 64 x 64.





Additional Data

id	PredictionString
00188a671292_stud	negative 1 0 0 1 1
004bd59708be_stud	negative 1 0 0 1 1
00508faccd39_study	negative 1 0 0 1 1
006486aa80b2_stud	negative 1 0 0 1 1
00655178fdfc_study	negative 1 0 0 1 1
00a81e8f1051_study	negative 1 0 0 1 1
00be7de16711_stud	negative 1 0 0 1 1
00c7a3928f0f_study	negative 1 0 0 1 1
00d63957bc3a_stud	negative 1 0 0 1 1
0107f2d291d6_study	negative 1 0 0 1 1
0154653179fa_study	negative 1 0 0 1 1
015f89ec55ea_study	negative 1 0 0 1 1
0241bc13eac6_stud	negative 1 0 0 1 1

id	Negative for Pneumo	Typical Appearance	Indeterminate Appea	Atypical Appearance
00086460a852_stud	0	1	0	0
000c9c05fd14_study	0	0	0	1
00292f8c37bd_study	1	0	0	0
005057b3f880_study	1	0	0	0

Convolutional Neural Networks

What are Convolutional Neural Networks?

Convolutional neural networks use three-dimensional data to for image classification and object recognition tasks.

1. Place small filter (kernel) on top of image patch
2. Multiply overlapping values and sum them up
3. Slide filter across image, repeat at each location
4. Output is a new image highlighting certain features (edges, textures, etc)
5. Convolution takes in an image and outputs a transformed image

$$(W * I)(i,j) = \sum_{p,q=-N}^{N} W(N+1+p, N+1+q) I(i+p, j+q)$$

Model Code - Construction

Code - Setting up Classes & CNN's

```python
# Custom dataset for binary COVID classification
class CustomNumpyDataset(Dataset):
    def __init__(self, data_array, labels_array):
        self.data = torch.from_numpy(data_array).float()
        self.labels = torch.from_numpy(labels_array).long()

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        sample = self.data[idx].unsqueeze(0)  # Add channel dimension
        label_idx = torch.argmax(self.labels[idx]).item()
        binary_label = 1 if label_idx in [2, 3] else 0  # 1: COVID, 0: No-COVID
        return sample, binary_label
```

```python
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        w = (32, 64, 128, 256)
        self.conv1 = nn.Conv2d(1, w[0], 3, 1)
        self.bn1 = nn.BatchNorm2d(w[0])
        self.conv2 = nn.Conv2d(w[0], w[1], 3, 1)
        self.bn2 = nn.BatchNorm2d(w[1])
        self.conv3 = nn.Conv2d(w[1], w[2], 3, 1)
        self.bn3 = nn.BatchNorm2d(w[2])
        self.conv4 = nn.Conv2d(w[2], w[3], 3, 1)
        self.bn4 = nn.BatchNorm2d(w[3])

        self._to_linear = None
        self._get_linear_input_size()

        self.fc1 = nn.Linear(self._to_linear, 512)
        self.dropout = nn.Dropout(p=0.7)
        self.fc2 = nn.Linear(512, 2)  # Binary classification
```

Code - Training & Testing

```python
def train(model, device, train_loader, optimizer, epoch):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = F.nll_loss(output, target)
        loss.backward()
        optimizer.step()
        if batch_idx % 20 == 0:
            print(f'Train Epoch: {epoch} [{batch_idx * len(data)}/{len(train_loader.dataset)}
                  f'({100. * batch_idx / len(train_loader):.0f}%)]\tLoss: {loss.item():.6f}')
```

```python
def test(model, device, test_loader, scheduler):
    model.eval()
    test_loss = 0
    correct = 0
    total = 0
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            test_loss += F.nll_loss(output, target, reduction='sum').item()
            pred = output.argmax(dim=1, keepdim=True)
            correct += pred.eq(target.view_as(pred)).sum().item()
            total += len(target)
```

Code - Training Settings

```python
# Train/test split
train_images, test_images, train_labels, test_labels = train_test_split(images, labels, test_size=0.3, random_state=42)
train_ds = CustomNumpyDataset(train_images, train_labels)
test_ds = CustomNumpyDataset(test_images, test_labels)

# DataLoaders
batch_size = 100
test_batch_size = 500

train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True, num_workers=1, pin_memory=True)
test_loader = DataLoader(test_ds, batch_size=test_batch_size, shuffle=False, num_workers=1, pin_memory=True)
```

```python
# Initialize model, optimizer, scheduler
model = Net().to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = ReduceLROnPlateau(optimizer, 'min', patience=2, factor=0.6)

# Training loop
epochs = 10  # You can increase for better performance
for epoch in range(1, epochs + 1):
    train(model, device, train_loader, optimizer, epoch)
    acc = test(model, device, test_loader, scheduler)
```

Conclusion

Final Accuracy

Training Settings & BatchNorms played a huge role, allowing us to increase accuracy from 47% to 75%.

```
Final Test Accuracy: 75.07%
```

Future improvements

- Improving accuracy by accounting for other factors
- Increasing the amount of data to account for diversity
- Incorporate time-series models to take into account the progression of lung health
- Account for any bias present in the dataset

Thank You For Listening

Any Questions?